澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

Introduction to Numeric Computing

Jie Zhang
Faculty of Applied Sciences, Macao Polytechnic University
Email: jpeter.zhang@mpu.edu.mo
Personal Profile: easy-shu.github.io



Python Packages for Data Science

- Numpy and Scipy-fundamental scientific computing
- Pandas-data manipulation and analysis.
- Matplotlib, Seaborn, and Plotnine - plotting and visualization.
- Scikit-learn- machine learning and data mining
- StatsModels-statistical modeling, testing, and analysis



Install Documentation Learn Community About Us News Contribute English

NumPy

The fundamental package for scientific computing with Python

LATEST RELEASE: NUMPY 2.2. VIEW ALL RELEASES

NumPy 2.2.0 released! 2024-12-08

Powerful N-dimensional arrays

Fast and versatile, the NumPy vectorization, indexing, and broadcasting concepts are the de-facto standards of array computing today.

Numerical computing tools

NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more.

Open source

Distributed under a liberal BSD license, NumPy is developed and maintained publicly on GitHub by a vibrant, responsive, and diverse community.

Interoperable

NumPy supports a wide range of hardware and computing platforms, and plays well with distributed, GPU, and sparse array libraries.

Performant

The core of NumPy is well-optimized C code. Enjoy the flexibility of Python with the speed of compiled code.

Easy to use

NumPy's high level syntax makes it accessible and productive for programmers from any background or experience level.

Try NumPy

Use the interactive shell to try NumPy in the browser

https://numpy.org/



Outline

1. Array Creation
2. Array Reading and Writing
3. Array Subsetting, Slicing, Indexing
4. Array Manipulation
5. Array Mathematics
6. Array Analysis



Numpy

NumPy is a fundamental library in Python that plays a crucial role in high-performance computing and data analysis.

- It leverages low-level implementations in C an FORTRAN to provide efficient mathematical functions, outperforming slower Python algorithms.
- By utilizing multidimensional arrays and vectorized operations, NumPy enables streamlined computations, ensuring speed and efficiency.

- **Conda:** If you use conda, you can install NumPy from the defaults or conda-forge channels:

```
conda create -n my-env
conda activate my-env
conda install numpy
```

- **Pip:**

```
pip install numpy
```



Numpy

NumPy is a fundamental library in Python that plays a crucial role in high-performance computing and data analysis.

- It leverages low-level implementations in C an FORTRAN to provide efficient mathematical functions, outperforming slower Python algorithms.
- By utilizing multidimensional arrays and vectorized operations, NumPy enables streamlined computations, ensuring speed and efficiency.

Verifying the Installation

After installing NumPy, verify the installation by running the following in a Python shell or script:

```
import numpy as np
print(np.__version__)
```




Numpy

- It leverages low-level implementations in C an FORTRAN to provide efficient mathematical functions, outperforming slower Python algorithms.

python ▶ 运行 复制

```python
import numpy as np
import time

# Generating a large array
big_array = np.random.rand(1000000)

# Using NumPy for sum computation and measuring time
start_time = time.time()
result_numpy = np.sum(big_array)
numpy_time = time.time() - start_time
print("NumPy time:", numpy_time)

# Using Python's built-in sum() for comparison and measuring time
start_time = time.time()
result_python = sum(big_array)
python_time = time.time() - start_time
print("Python time:", python_time)
```



Numpy

- By utilizing multidimensional arrays and vectorized operations, NumPy enables streamlined computations, ensuring speed and efficiency.

python ▶ 运行 复制

```python
import numpy as np
import time

# Using NumPy for element-wise multiplication and measuring time
start_time = time.time()
arr1 = np.random.rand(1000, 1000)
arr2 = np.random.rand(1000, 1000)
result_numpy = arr1 * arr2
numpy_time = time.time() - start_time

print("NumPy time:", numpy_time)

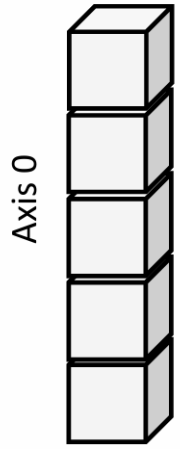
# Using Python lists for element-wise multiplication and measuring time
start_time = time.time()
list1 = [[elem for elem in row] for row in arr1]
list2 = [[elem for elem in row] for row in arr2]
result_list = [[list1[i][j] * list2[i][j] for j in range(len(list1[0]))] for i in
range(len(list1))]
python_time = time.time() - start_time

print("Python time:", python_time)
```



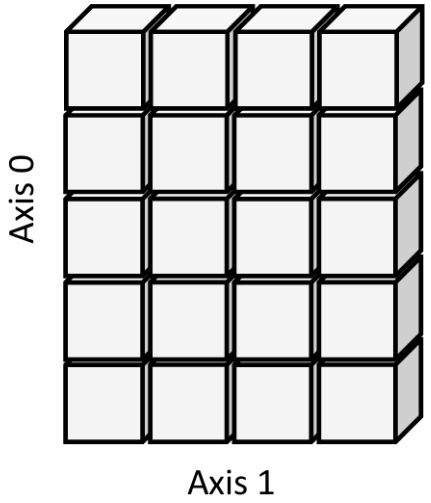

NumPy Arrays

1D NumPy Array



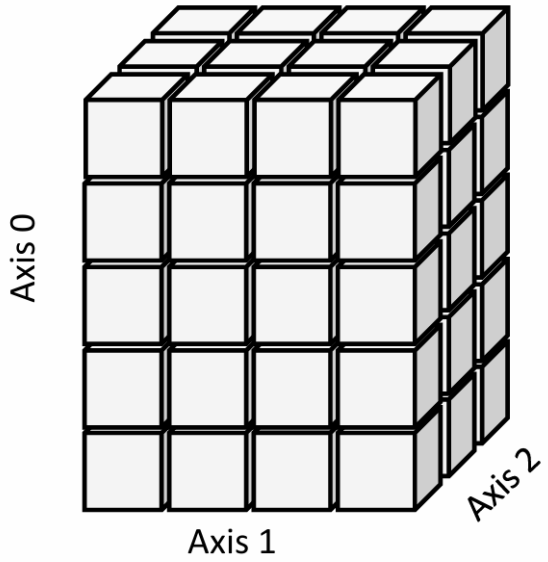
→ a.ndim = 1 *„axis 0“*
→ a.shape = (5,) *„five rows“*
→ a.size = 5 *„5 elements“*

2D NumPy Array



→ a.ndim = 2 *„axis 0 and axis 1“*
→ a.shape = (5, 4) *„five rows, four cols“*
→ a.size = 20 *„5*4=20 elements“*

3D NumPy Array



→ a.ndim = 3 *„axis 0 and axis 1“*
→ a.shape = (5, 4, 3) *„5 rows, 4 cols, 3 levels“*
→ a.size = 60 *„5*4*3=60 elements“*

Array Creation: Creating Arrays from Lists

Creating arrays in NumPy is simple and straightforward. You can create arrays from lists or tuples using the numpy.array() function:

```
import numpy as np
a = np.array([1, 2, 3]) # Creates a 1D array
b = np.array([(1, 2, 3), (4, 5, 6)]) # Creates a 2D array
```

Initialize a multidimensional array using a list of lists

```
In[11]: # nested lists result in multidimensional arrays
        np.array([range(i, i + 3) for i in [2, 4, 6]])

Out[11]: array([[2, 3, 4],
                [4, 5, 6],
                [6, 7, 8]])
```

NumPy provides several data types to store data in arrays, such as integer, string, float, boolean and complex. By default, NumPy tries to deduce the data type based on the input elements.

However, you can also explicitly specify the data type using the dtype keyword. For example:

```
import numpy as np
a = np.array([1, 2, 3], dtype=float)
# Creates an array of floats
```

Common NumPy data types include:

- np.int32: 32-bit integer
- np.int64: 64-bit integer
- np.float32: 32-bit floating-point number
- np.float64: 64-bit floating-point number
- np.complex: Complex number, represented by two 64-bit floating-point numbers

You can also convert arrays from one data type to another. In this example, here's how we can convert the Integer array a into a Boolean array **arr** using the **np.array()** method.



Array Creation: Creating Arrays from Lists

NumPy provides several data types to store data in arrays, such as integer, string, float, boolean and complex. By default, NumPy tries to deduce the data type based on the input elements.

However, you can also explicitly specify the data type using the dtype keyword. For example:

```
import numpy as np
a = np.array([1, 2, 3], dtype=float)
# Creates an array of floats
```

Common NumPy data types include:

- np.int32: 32-bit integer
- np.int64: 64-bit integer
- np.float32: 32-bit floating-point number
- np.float64: 64-bit floating-point number
- np.complex: Complex number, represented by two 64-bit floating-point numbers

You can also convert arrays from one data type to another. In this example, here's how we can convert the Integer array a into a Boolean array **arr** using the **np.array()** method.

Table *Standard NumPy data types*

Data type	Description
`bool_`	Boolean (True or False) stored as a byte
`int_`	Default integer type (same as C `long`; normally either `int64` or `int32`)
`intc`	Identical to C `int` (normally `int32` or `int64`)
`intp`	Integer used for indexing (same as C `ssize_t`; normally either `int32` or `int64`)
`int8`	Byte (–128 to 127)
`int16`	Integer (–32768 to 32767)
`int32`	Integer (–2147483648 to 2147483647)
`int64`	Integer (–9223372036854775808 to 9223372036854775807)
`uint8`	Unsigned integer (0 to 255)
`uint16`	Unsigned integer (0 to 65535)
`uint32`	Unsigned integer (0 to 4294967295)
`uint64`	Unsigned integer (0 to 18446744073709551615)
`float_`	Shorthand for `float64`
`float16`	Half-precision float: sign bit, 5 bits exponent, 10 bits mantissa
`float32`	Single-precision float: sign bit, 8 bits exponent, 23 bits mantissa
`float64`	Double-precision float: sign bit, 11 bits exponent, 52 bits mantissa
`complex_`	Shorthand for `complex128`
`complex64`	Complex number, represented by two 32-bit floats
`complex128`	Complex number, represented by two 64-bit floats



Array Creation: Creating Arrays from Scratch

Input	Output	Description
np.arange(0,3,1)	array([0, 1, 2])	Arithmetic progression with step size 1
np.linspace(0,3,4)	array([0., 1., 2., 3.])	Linearly spaced array with 4 elements
np.repeat([1,2],2)	array([1, 1, 2, 2])	Replicate and tile array elements
np.tile([1,2],2)	array([1, 2, 1, 2])	Replicate and tile array elements
np.ones((2,3))	array([[1., 1., 1.], [1., 1., 1.]])	Create an array of ones
np.zeros((2,3))	array([[0., 0., 0.], [0., 0., 0.]])	Create an array of zeros
np.random.random(3)	array([0.24, 0.74, 0.95])	Random numbers between 0 and 1
np.random.randn(3)	array([-1.44, 0.39, 1.8])	Standard normal distribution with mean 0 and standard deviation 1
np.random.randint(0, 10, (3, 3))	array([[6, 7, 4], [6, 8, 1], [1, 5, 9]])	Create a 3x3 array of random integers in the interval [0, 10)
np.random.normal(loc=0,scale=1,size=3)	array([0.55, -2.03, -0.21])	Normal distribution with mean 0 and standard deviation 1
np.empty(3)	array([0.93229253, 0.9144306 , 0.97442155])	create the required array shape and fills it with random values. The default method creates an array of random floats.



Array Creation: Creating Arrays from Scratch

Input	Output	Description
np.ones_like()	array([1., 1., 1.])	Create an array of ones with the same shape as the input array
np.zeros_like()	array([0., 0., 0.])	Create an array of zeros with the same shape as the input array
np.full((2, 3), 5)	array([[5, 5, 5], [5, 5, 5]])	Create a constant array filled with a specified value
np.eye(3)	array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]])	Create an identity matrix
np.diag([1, 2, 3])	array([[1, 0, 0], [0, 2, 0], [0, 0, 3]])	Create a diagonal matrix with specified diagonal elements
np.random.randint(1, 10, size=(2, 2))	array([[6, 8], [2, 9]])	Generate random integers within a specified range
np.random.uniform(0, 1, size=3)	array([0.3, 0.6, 0.8])	Generate random numbers from a uniform distribution
np.random.choice([1, 2, 3, 4], size=3)	array([2, 4, 1])	Randomly choose elements from a given array
np.unique([1, 2, 1, 3, 2])	array([1, 2, 3])	Find the unique elements in an array and return them sorted



Array Creation: Creating Arrays from Scratch

```
In[1]: import numpy as np
       np.random.seed(0)  # seed for reproducibility
       x1 = np.random.randint(10, size=6)  # One-dimensional array
       x2 = np.random.randint(10, size=(3, 4))  # Two-dimensional array
       x3 = np.random.randint(10, size=(3, 4, 5))  # Three-dimensional array
```

```
In[2]: print("x3 ndim: ", x3.ndim)
       print("x3 shape:", x3.shape)
       print("x3 size: ", x3.size)
```

```
x3 ndim:  3
x3 shape: (3, 4, 5)
x3 size:  60
```

Inspecting Your Array

#Convert an array to a different type
b=a.astype(float)
Length of array
Length=len(a)

Inspecting Your Array

NumPy arrays have several attributes that provide useful information about the array. Let's look at some of them:

- **ndarray.shape:** Returns the dimensions of the array as a tuple (rows, columns)
- **ndarray.ndim:** Returns the number of dimensions in the array
- **ndarray.size:** Returns the total number of elements in the array
- **ndarray.dtype:** Returns the data type of the array elements

To access these attributes, use the dot notation, like so:

```
a = np.array([(1, 2, 3), (4, 5, 6)])
#Print out the array shape
print(a.shape) # Output: (2, 3)
```



Array Reading and Writing

1. Reading and Writing NumPy Arrays from/to Text Files:

python ▶ 运行 复制

```python
import numpy as np

# Creating a sample NumPy array
data = np.array([[1, 2, 3], [4, 5, 6]])

# Saving the NumPy array to a text file
np.savetxt('data.txt', data)

# Loading data from the text file into a NumPy array
loaded_data = np.loadtxt('data.txt')
print(loaded_data)
```




Array Reading and Writing

2. Reading and Writing NumPy Arrays in Binary Format:

python ▶ 运行 复制

```python
import numpy as np

# Creating a sample NumPy array
data = np.array([[1, 2, 3], [4, 5, 6]])

# Saving the NumPy array to a binary file
np.save('data.npy', data)

# Loading data from the binary file into a NumPy array
loaded_data = np.load('data.npy')
print(loaded_data)
```




Array Reading and Writing

3. Reading and Writing NumPy Arrays from/to CSV Files:

python ▶ 运行 复制

```python
import numpy as np

# Creating a sample NumPy array
data = np.array([[1, 2, 3], [4, 5, 6]])

# Saving the NumPy array to a CSV file
np.savetxt('data.csv', data, delimiter=',')

# Loading data from the CSV file into a NumPy array
loaded_data = np.loadtxt('data.csv', delimiter=',')
print(loaded_data)
```



Array Subsetting, Slicing, Indexing: Accessing Single Elements

1D array

```
x1=np.array([5, 0, 3, 3, 7, 9])

x1[0]
x1[4]
x1[-1]
x1[-2]
```

```
array([5, 0, 3, 3, 7, 9])
```

2D array

```
x2=np. array([[3, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 7]])

x2[0, 0]
x2[2, 0]
x2[2, -1]
x2[0, 0]
```

```
array([[3, 5, 2, 4],
       [7, 6, 8, 8],
       [1, 6, 7, 7]])
```

Keep in mind that, unlike Python lists, NumPy arrays have a fixed type. This means, for example, that if you attempt to insert a floating-point value to an integer array, the value will be silently truncated.

Don’t be caught unaware by this behavior!

```
x1=np.array([5, 0, 3, 3, 7, 9])
x1[0] = 3.14159 # this will be truncated!
print(x1)
```

```
x1=np.array([5, 0, 3, 3, 7, 9],dtype=float)
x1[0] = 3.14159
print(x1)
```



Array Subsetting, Slicing, Indexing: Accessing Subarrays

The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array x, use this:

x[start:stop:step]

If any of these are unspecified, they default to the values start=0, stop=size of dimension, step=1.

One-dimensional subarrays

```
x = np.arange(10)      # array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
x[:5] # first five elements
x[5:] # elements after index 5
x[4:7] # middle subarray
x[::2] # every other element
x[1::2] # every other element, starting at index 1
x[::-1] # all elements, reversed
x[5::-2] # reversed every other from index 5
x[x>3]   #Boolean Indexing: Select elements from a less than 3
X[(x>3) & (x<8)]
```



Array Subsetting, Slicing, Indexing: Accessing Subarrays

The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array x, use this:

x[start:stop:step]

If any of these are unspecified, they default to the values start=0, stop=size of dimension, step=1.

One-dimensional subarrays

```
x = np.arange(10)          # array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
x[:5] # first five elements                          #array([0, 1, 2, 3, 4])
x[5:] # elements after index 5                       #array([5, 6, 7, 8, 9])
x[4:7] # middle subarray                             #array([4, 5, 6])
x[::2] # every other element                         #array([0, 2, 4, 6, 8])
x[1::2] # every other element, starting at index 1   #array([1, 3, 5, 7, 9])
x[::-1] # all elements, reversed                     #array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
x[5::-2] # reversed every other from index 5         #array([5, 3, 1])
x[x>3]  #Boolean Indexing: Select elements from a less than 3
X[(x>3) & (x<8)]
```



Array Slicing: Accessing Rows and Columns, and Subarrays

The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array x, use this:

x[start:stop:step]

If any of these are unspecified, they default to the values start=0, stop=size of dimension, step=1.

Multidimensional subarrays	x2 = np. array([[12, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 7]])

```
x2[1,2]   x2[0,-1]    x2[-1,-1]     x2[-3,-1]

x2[:, 0]   # first column of x2
x2[:,[0,1]]
x2[:,:2]
x2[:,1:2]

x2[0, :] # first row of x2
x2[0] # equivalent to x2[0, :]
x2[[0,1],:]
x2[:2,:]
x2[1:2,:]
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```



Array Subsetting, Slicing, Indexing: Accessing Rows and Columns, and Subarrays

The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array x, use this:

x[start:stop:step]

If any of these are unspecified, they default to the values start=0, stop=size of dimension, step=1.

Multidimensional subarrays	x2 = np. array([[12, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 7]])

x2[:2, :3] # two rows, three columns

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```

x2[:3, ::2] # all rows, every other column

x2[::-1, ::-1] #subarray dimensions can be reversed together

x2[[1,0,1,0],[0,1,2,0]] # Fancy Indexing: Select elements (1,0),(0,1),(1,2) and (0,0)

b[[1, 0, 1, 0]][:,[0,1,2,0]] # Fancy Indexing: Select a subset of the matrix's rows and columns



Array Subsetting, Slicing, Indexing: Accessing Rows and Columns, and Subarrays

The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array x, use this:

x[start:stop:step]

If any of these are unspecified, they default to the values start=0, stop=size of dimension, step=1.

Multidimensional subarrays x2 = np. array([[12, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 7]])

x2[:2, :3] # two rows, three columns

```
array([[12,  5,  2],
       [ 7,  6,  8]])
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```

x2[:3, ::2] # all rows, every other column

```
array([[12,  2],
       [ 7,  8],
       [ 1,  7]])
```

x2[::-1, ::-1] #subarray dimensions can be reversed together

```
array([[ 7,  7,  6,  1],
       [ 8,  8,  6,  7],
       [ 4,  2,  5, 12]])
```

x2[[1,0,1,0],[0,1,2,0]] # Fancy Indexing: Select elements (1,0),(0,1),(1,2) and (0,0)

b[[1, 0, 1, 0]][:,[0,1,2,0]] # Fancy Indexing: Select a subset of the matrix's rows and columns



Array Manipulation: Coping-Subarrays as no-copy view

One important—and extremely useful—thing to know about array slices is that they return views rather than copies of the array data.

This is one area in which NumPy array slicing differs from Python list slicing: in lists, slices will be copies.

Multidimensional subarrays	x2 = np. array([[12, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 7]])

```
x2 = np. array([[12, 5, 2, 4], [ 7, 6, 8, 8], [ 1, 6, 7, 7]])
x2_sub = x2[:2, :2]
print(x2_sub)

x2_sub[0, 0] = 99
print(x2_sub)

print(x2)
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```

```
#Creating a shallow copy of a NumPy array
a = np.array([9, 6, 12, 16, 20])
b = a
b[0] = 19
print(a) #Output:[19, 6, 12, 16, 20]
print(b) #Output:[19, 6, 12, 16, 20]
```




Array Manipulation: Coping-Subarrays as no-copy view

One important—and extremely useful—thing to know about array slices is that they return views rather than copies of the array data.

This is one area in which NumPy array slicing differs from Python list slicing: in lists, slices will be copies.

Multidimensional subarrays	x2 = np. array([[12, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 7]])

```
x2 = np. array([[12, 5, 2, 4], [ 7, 6, 8, 8], [ 1, 6, 7, 7]])
x2_sub = x2[:2, :2]
print(x2_sub)
```

```
[[12  5]
 [ 7  6]]
```

```
x2_sub[0, 0] = 99
print(x2_sub)
```

```
[[99  5]
 [ 7  6]]
```

```
print(x2)
```

```
[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```



Array Manipulation: Coping-Creating copies of arrays

Despite the nice features of array views, it is sometimes useful to instead explicitly copy the data within an array or a subarray. This can be most easily done with the copy() method

Multidimensional subarrays	x2 = np. array([[12, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 7]])

```
x2 = np. array([[12, 5, 2, 4], [ 7, 6, 8, 8], [ 1, 6, 7, 7]])
x2_sub_copy = x2[:2, :2].copy()   #x2_sub_copy = np. copy(x2[:2, :2])
print(x2_sub)

x2_sub_copy[0, 0] = 42
print(x2_sub_copy)

print(x2)
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```

```
#Creating a deep copy of a NumPy array
a = np.array([9, 6, 12, 16, 20])
b = np.copy(a)
b[0] = 19
print(a) #Output:[9, 6, 12, 16, 20]
print(b) #Output:[19, 6, 12, 16, 20]
```




Array Manipulation: Coping-Creating copies of arrays

Despite the nice features of array views, it is sometimes useful to instead explicitly copy the data within an array or a subarray. This can be most easily done with the copy() method

Multidimensional subarrays x2 = np. array([[12, 5, 2, 4], [7, 6, 8, 8], [1, 6, 7, 7]])

```
x2 = np. array([[12, 5, 2, 4], [ 7, 6, 8, 8], [ 1, 6, 7, 7]])
x2_sub_copy = x2[:2, :2].copy()
print(x2_sub)
```

```
[[12  5]
 [ 7  6]]
```

```
x2_sub_copy[0, 0] = 42
print(x2_sub_copy)
```

```
[[42  5]
 [ 7  6]]
```

```
print(x2)
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```



Array Manipulation: Reshaping

Reshaping an array in NumUy is a common task you will perform. You might need to change the shape of your array to match the requirements of a function or an algorithm.

```
arr = np.array([1, 2, 3, 4, 5,6])

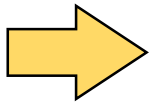
a_arr = arr.reshape(2, 3)
b_arr=arr.reshape(-1,3)

New_arr1=arr[np.newaxis, :]    # row vector via newaxis
New_arr2=arr[:, np.newaxis]    # column vector via newaxis
New_arr3= arr [None,:]         # row vector via newaxis
New_arr4=arr[:,None]           # column vector via newaxis
```

```
arr2=np.arange(6).reshape(3,2)

a_arr2= arr2.flatten()  #Flatten the array and return a new copy of the array
b_arr2= arr2.ravel()    #Flatten the array and  return a new view of the array
```

	0	1
0	0	1
1	2	3
2	4	5



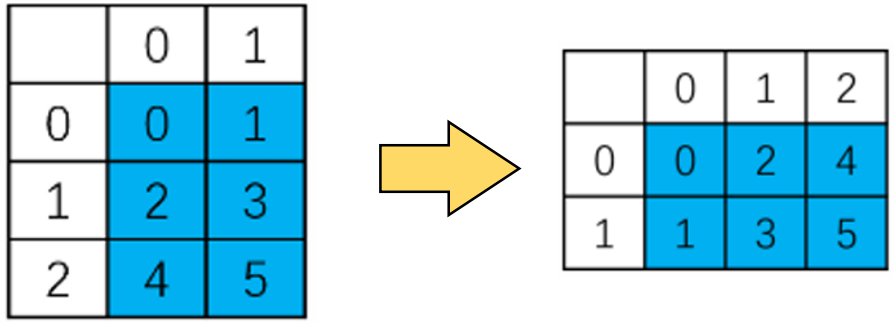
	0
0	0
1	1
2	2
3	3
4	4
5	5



Array Manipulation: Transposing

arr2=np.arange(6).reshape(3,2)

a_arr2= arr2.T # Permute array dimensions
b_arr2=np.transpose(arr2) # Permute array dimensions




```python
import numpy as np

# Create a sample 3D NumPy array with dimensions 0-1-2
array_0_1_2 = np.array([[[1, 2], [3, 4]],
                        [[5, 6], [7, 8]],
                        [[9, 10], [11, 12]]])

# Transpose the array to change the dimensions to 2-0-1
array_2_0_1 = np.transpose(array_0_1_2, (2, 0, 1))

print("Original Array (0-1-2):")
print(array_0_1_2.shape, array_0_1_2[:,0,0])

print("\nTransposed Array (2-0-1):")
print(array_2_0_1.shape,array_2_0_1[0,:,0])
```

```
Original Array (0-1-2):
(3, 2, 2) [1 5 9]

Transposed Array (2-0-1):
(2, 3, 2) [1 5 9]
```

Array Manipulation: Adding/Removing Elements

- np.append() is used to add a row or column to a 2D NumPy array.
- np.delete() is used to remove a row or column from a 2D NumPy array.

Using np.append() and np.delete() with 1D Arrays:

```
arr = np.array([1, 2,3])
arr = np.append(arr, [4, 5, 6]) # Appends values to theend of the array
arr =np.delete(arr,0)    # Removes the array element onindex 0
```

Using np.append() with 2D Arrays:

```
# Create a sample 2D NumPy array
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

# Append a row to the array
new_row = np.array([[7, 8, 9]])
arr_appended = np.append(arr, new_row, axis=0)

# Append a column to the array
new_col = np.array([[10], [11]])
arr_appended_col = np.append(arr, new_col, axis=1)
```

Using np.delete() with 2D Arrays

```
# Create a sample 2D NumPy array
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

# Delete a row from the array
arr_deleted_row = np.delete(arr, 1, axis=0)

# Delete a column from the array
arr_deleted_col = np.delete(arr, 1, axis=1)
```



Array Manipulation: Adding/Removing Elements

- resize(): The resize() function modifies the array itself to the specified shape. If the new size is larger than the original size, the original array will be repeated or filled with zeros. If the new size is smaller, the array will be truncated. This function changes the array in place.
- reshape(): The reshape() function returns a new view of the array with the specified shape, without modifying the original array. If the new shape is not compatible with the original shape (i.e., the total number of elements is not the same), it will raise an error.

```
# Create a sample NumPy array
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Using resize()
arr_resize = np.resize(arr, (3, 3))

# Modify the array in place # Using reshape()
arr_reshape1 = arr.reshape((3, 2)) # Return a new view of the array
arr_reshape2 = np.reshape(arr, (3, 2)) # Return a new view of the array
```



Array Manipulation: Array Concatenation

Concatenation, or joining of two arrays in NumPy, is primarily accomplished through the routines np.concatenate, np.vstack, and np.hstack. np.concatenate takes a tuple or list of arrays as its first argument,

a=np.arange(6).reshape(3,2)

	0	1
0	0	1
1	2	3
2	4	5

b=np.arange(9).reshape(3,3)

	0	1	2
0	0	1	2
1	3	4	5
2	6	7	8

c=np.arange(6).reshape(2,3)

	0	1	2
0	0	1	2
1	3	4	5

Horizontal combinations of arrays:

	0	1	2	3	4
0	0	1	0	1	2
1	2	3	3	4	5
2	4	5	6	7	8

- np.hstack((a,b)) **(row-wise)**
- np.concatenate((a,b),axis=1)
- np.append(a,b,axis=1)
- np.c_[a,b]

Vertical combinations of arrays:

	0	1	2
0	0	1	2
1	3	4	5
2	6	7	8
3	0	1	2
4	3	4	5

- np.vstack((b,c)) **(column-wise)**
- np.concatenate((b,c),axis=0)
- np.append(b,c,axis=0)
- np.r_[b,c]



Array Manipulation: Array Splitting

The opposite of concatenation is splitting, which is implemented by the functions np.split, np.hsplit, and np.vsplit.

```
a = np.array([1,2,3])
c = np.array([[(1.5,2,3), (4,5,6)],
              [(3, 2, 1), (4,5,6)]],
              dtype = float)       #c.shape: (2, 2, 3)

A1=np.hsplit(a,3)   #Split the array horizontally at the 3rd index
#[array([1]),array([2]),array([3])]

A2=np.split(a, [2,1])   #Pass a list of indices giving the split points

C1=np.vsplit(c,2)   #Split the array vertically at the 2nd index
#[array([[[ 1.5, 2. , 1. ], [ 4. , 5. , 6. ]]]),
# array([[[ 3., 2., 3.],      [ 4., 5., 6.]]])]

C2= np.split(c, 2, axis=0)    # Split the array into 4 sub-arrays along axis 0
C3= np.split(c, 2, axis=1)    # Split the array into 3 sub-arrays along axis 1
```




Array Mathematics: Comparisons, Masks, and Boolean Logic

```
# 1D Array
A1 = np.array([1, 2, 3, 4, 5]) # Create a sample array

comparison1 = A1 > 2   # Element-wise comparison
masked_array1a = A1[comparison1] # Applying the mask based on the condition
masked_array1b= A1[A1 > 2 ] # Applying the mask based on the condition

masked_array1c= A1[(A1 > 2) & (A1 > 4) ]  #Multiple conditions
```

```
# 2D Array
A2 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Create a sample 2D array

comparison2 = A2 > 2   # Element-wise comparison
masked_array2a = A2[comparison2] # Applying the mask based on the condition
masked_array2b = A2[A2 > 2 ] # Applying the mask based on the condition

masked_array2c = A2[(A2 > 2 ) & (A2 < 5)]  #Multiple conditions
```

Operator	Equivalent ufunc
==	np.equal
!=	np.not_equal
<	np.less
<=	np.less_equal
>	np.greater
>=	np.greater_equal
&	np.bitwise_and
|	np.bitwise_or
^	np.bitwise_xor
~	np.bitwise_not



Array Mathematics: Arithmetic Operations

- a = np.array([1,2,3])
- b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
- c = np.array([[(1.5,2,3), (4,5,6)], [(3,2,1), (4,5,6)]], dtype = float)

```
>>> g = a - b                                  Subtraction
  array([[-0.5,  0. ,  0. ],
         [-3. , -3. , -3. ]])
>>> np.subtract(a,b)                           Subtraction
>>> b + a                                      Addition
  array([[ 2.5,  4. ,  6. ],
         [ 5. ,  7. ,  9. ]])
>>> np.add(b,a)                                Addition
>>> a / b                                      Division
  array([[ 0.66666667,  1.        ,  1.        ],
         [ 0.25      ,  0.4       ,  0.5       ]])
>>> np.divide(a,b)                             Division
>>> a * b                                      Multiplication
  array([[  1.5,   4. ,   9. ],
         [  4. ,  10. ,  18. ]])
>>> np.multiply(a,b)                           Multiplication
>>> np.exp(b)                                  Exponentiation
>>> np.sqrt(b)                                 Square root
>>> np.sin(a)                                  Print sines of an array
>>> np.cos(b)                                  Element-wise cosine
>>> np.log(a)                                  Element-wise natural logarithm
>>> e.dot(f)                                   Dot product
  array([[ 7.,  7.],
         [ 7.,  7.]])
```

NumPy offers various math operations on arrays that make them simple and efficient to work with. array mathematics vector math

Some of the operations are:

- **Addition:** numpy.add(x1, x2)
- **Subtraction:** numpy.subtract(x1, x2)
- **Multiplication:** numpy.multiply(x1, x2)
- **Division:** numpy.divide(x1, x2)
- **Modulus:** numpy.mod(x1, x2)
- **Power:** numpy.power(x1, x2)
- **Square root:** numpy.sqrt(x)

Note: When using these operations, the two arrays must be the same shape. If not, you'll run into errors.

There is an exception for certain arrays thanks to a NumPy feature called broadcasting. We'll cover that in a later section.

You can perform these operations element-wise on the arrays, which makes them highly efficient for large-scale data manipulation.



Array Mathematics: Arithmetic Operations

- a = np.array([1,2,3])
- b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
- c = np.array([[(1.5,2,3), (4,5,6)], [(3,2,1), (4,5,6)]], dtype = float)

```
>>> g = a - b                                  Subtraction
  array([[-0.5,  0. ,  0. ],
         [-3. , -3. , -3. ]])
>>> np.subtract(a,b)                           Subtraction
>>> b + a                                      Addition
  array([[ 2.5,  4. ,  6. ],
         [ 5. ,  7. ,  9. ]])
>>> np.add(b,a)                                Addition
>>> a / b                                      Division
  array([[ 0.66666667,  1.        ,  1.        ],
         [ 0.25      ,  0.4       ,  0.5       ]])
>>> np.divide(a,b)                             Division
>>> a * b                                      Multiplication
  array([[  1.5,   4. ,   9. ],
         [  4. ,  10. ,  18. ]])
>>> np.multiply(a,b)                           Multiplication
>>> np.exp(b)                                  Exponentiation
>>> np.sqrt(b)                                 Square root
>>> np.sin(a)                                  Print sines of an array
>>> np.cos(b)                                  Element-wise cosine
>>> np.log(a)                                  Element-wise natural logarithm
>>> e.dot(f)                                   Dot product
  array([[ 7.,  7.],
         [ 7.,  7.]])
```

Trigonometric functions play a significant role in various mathematical and scientific computations. NumPy provides a wide range of trigonometric functions.

Some of the essential functions are:

- **Sine:** numpy.sin(x)
- **Cosine:** numpy.cos(x)
- **Tangent:** numpy.tan(x)
- **Arcsine:** numpy.arcsin(x)
- **Arccosine:** numpy.arccos(x)
- **Arctangent:** numpy.arctan(x)

These functions work seamlessly with arrays, making it easier for you to perform vectorized computations on large datasets.



Array Mathematics: Arithmetic Operations

- a = np.array([1,2,3])
- b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
- c = np.array([[(1.5,2,3), (4,5,6)], [(3,2,1), (4,5,6)]], dtype = float)

```
>>> g = a - b                                  Subtraction
  array([[-0.5,  0. ,  0. ],
         [-3. , -3. , -3. ]])
>>> np.subtract(a,b)                           Subtraction
>>> b + a                                      Addition
  array([[ 2.5,  4. ,  6. ],
         [ 5. ,  7. ,  9. ]])
>>> np.add(b,a)                                Addition
>>> a / b                                      Division
  array([[ 0.66666667,  1.        ,  1.        ],
         [ 0.25      ,  0.4       ,  0.5       ]])
>>> np.divide(a,b)                             Division
>>> a * b                                      Multiplication
  array([[  1.5,   4. ,   9. ],
         [  4. ,  10. ,  18. ]])
>>> np.multiply(a,b)                           Multiplication
>>> np.exp(b)                                  Exponentiation
>>> np.sqrt(b)                                 Square root
>>> np.sin(a)                                  Print sines of an array
>>> np.cos(b)                                  Element-wise cosine
>>> np.log(a)                                  Element-wise natural logarithm
>>> e.dot(f)                                   Dot product
  array([[ 7.,  7.],
         [ 7.,  7.]])
```

Exponents and logarithms are crucial for various numerical operations. NumPy provides an extensive collection of functions for dealing with exponents and logarithms.

Some of the primary functions are:

- **Exponential:** numpy.exp(x)
- **Logarithm(base e):** numpy.log(x)
- **Logarithm(base 10):** numpy.log10(x)
- **Logarithm(base 2):** numpy.log2(x)

Utilizing these functions, you can quickly perform complex mathematical operations on each element in the array. This makes your data analysis tasks more accessible and efficient.



Array Mathematics: Broadcasting

Broadcasting is simply a set of rules for applying binary ufuncs (addition, subtraction, multiplication, etc.) on arrays of different sizes.

- Recall that for arrays of the same size, binary operations are performed on an element-by-element basis

```
a = np.array([0, 1, 2])
b = np.array([5, 5, 5])
c1=a+b
print(c1)
```

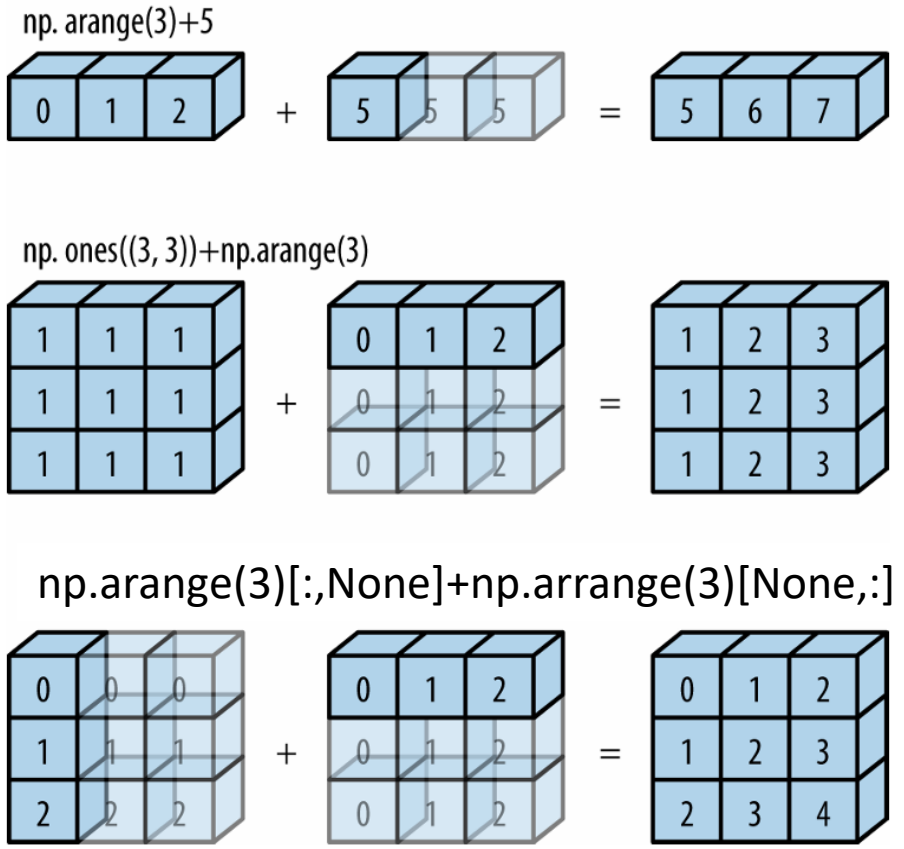
```
array([5, 6, 7])
```

- Broadcasting allows these types of binary operations to be performed on arrays of different sizes—for example, we can just as easily add a scalar (think of it as a zero dimensional array) to an array

```
a = np.array([0, 1, 2])
c2=a+5
print(c2)
```

```
array([5, 6, 7])
```

Visualization of NumPy broadcasting



Array Mathematics: Broadcasting

Broadcasting is simply a set of rules for applying binary ufuncs (addition, subtraction, multiplication, etc.) on arrays of different sizes.

```
a = np.array([1, 2, 3])
M = np.ones((3, 3))

C1=M+a
```

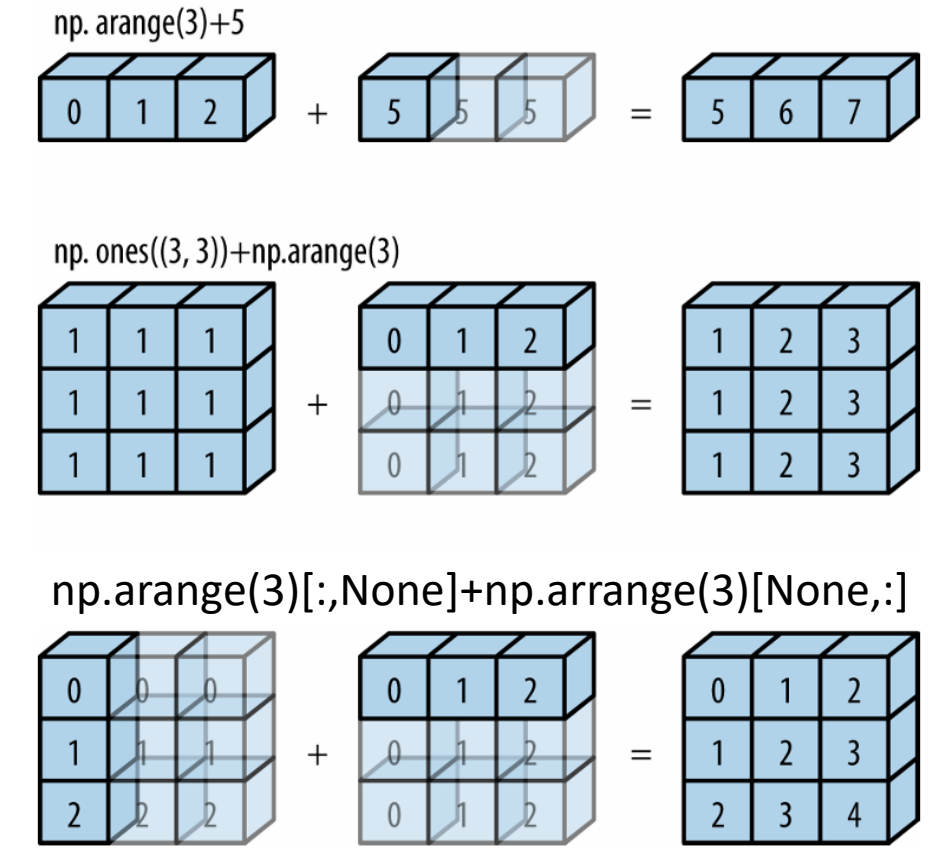
```
array([[2., 3., 4.],
       [2., 3., 4.],
       [2., 3., 4.]])
```

```
C2=M+a[None,:]
```

```
C3=M+a[:,None]
```

```
array([[2., 2., 2.],
       [3., 3., 3.],
       [4., 4., 4.]])
```

Visualization of NumPy broadcasting

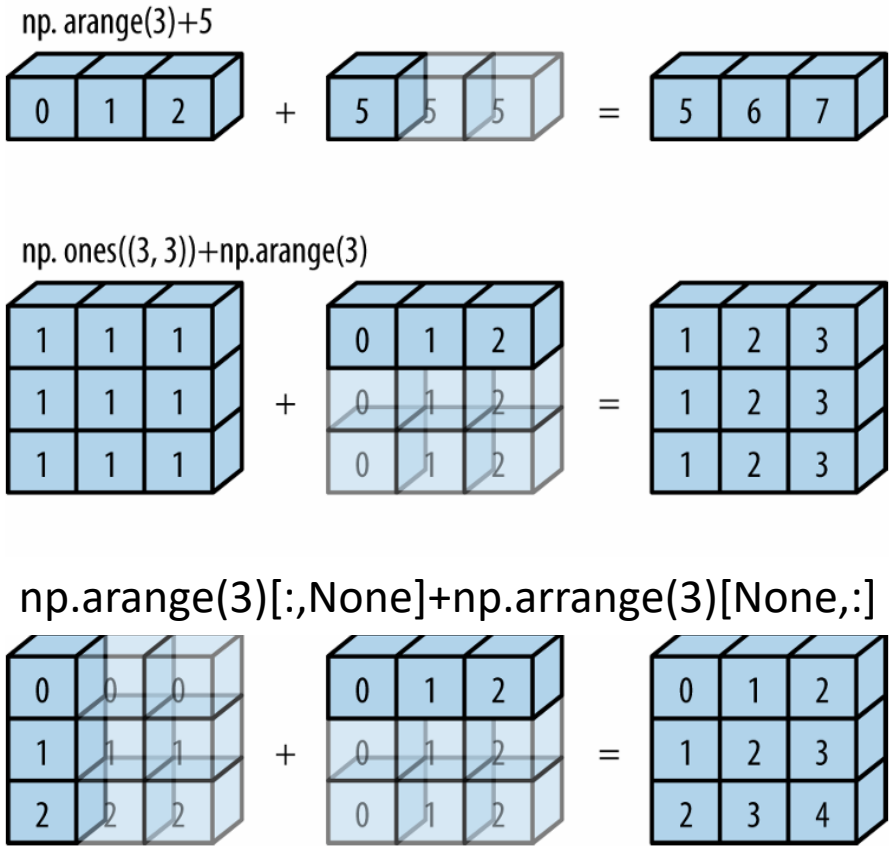


Array Mathematics: Broadcasting

Broadcasting in NumPy follows a strict set of rules to determine the interaction between the two arrays:

- Rule 1: If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is padded with ones on its leading (left) side.
- Rule 2: If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape.
- Rule 3: If in any dimension the sizes disagree and neither is equal to 1, an error is raised.

Visualization of NumPy broadcasting



Array Mathematics: Broadcasting

Broadcasting in NumPy follows a strict set of rules to determine the interaction between the two arrays:

```
M = np.ones((2, 3))   # M.shape = (2, 3)
a = np.arange(3)      # a.shape = (3,)
C=M+a                 #C.shape=(2, 3)
```

```
array([[ 1.,  2.,  3.],
       [ 1.,  2.,  3.]])
```

- We see by rule 1 that the array a has fewer dimensions, so we pad it on the left with ones:

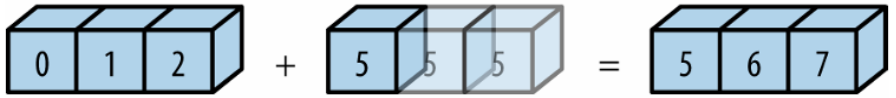
  M.shape -> (2, 3)
  a.shape -> (1, 3)

- By rule 2, we now see that the first dimension disagrees, so we stretch this dimension to match:

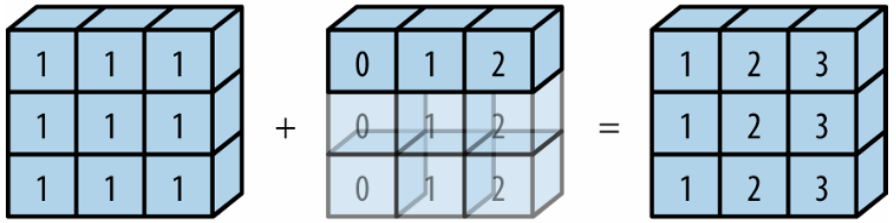
  M.shape -> (2, 3)
  a.shape -> (2, 3)

Visualization of NumPy broadcasting

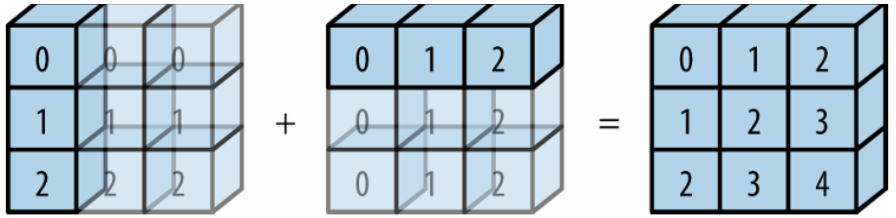
np. arange(3)+5



np. ones((3, 3))+np.arange(3)



np.arange(3)[:,None]+np.arrange(3)[None,:]



Array Mathematics: Broadcasting

Broadcasting in NumPy follows a strict set of rules to determine the interaction between the two arrays:

```
a = np.arange(3).reshape((3, 1))    # a.shape = (3,1)
b = np.arange(3)       # b.shape = (3,)
C=a+b                  #C.shape=(3, 3)
```

```
array([[0, 1, 2],
       [1, 2, 3],
       [2, 3, 4]])
```

- We see by rule 1 that the array a has fewer dimensions, so we pad it on the left with ones:

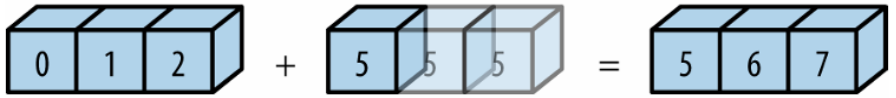
  a.shape -> (3, 1)
  b.shape -> (1, 3)

- By rule 2, we now see that the first dimension disagrees, so we stretch this dimension to match:

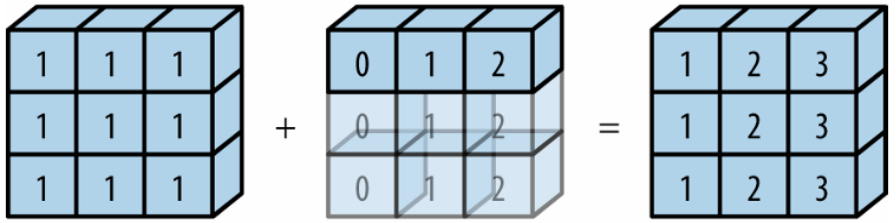
  a.shape -> (3, 3)
  b.shape -> (3, 3)

Visualization of NumPy broadcasting

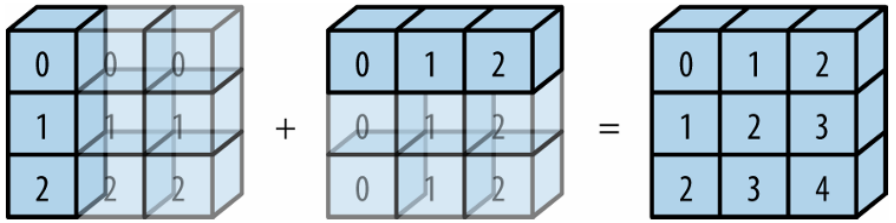
np. arange(3)+5



np. ones((3, 3))+np.arange(3)



np.arange(3)[:,None]+np.arrange(3)[None,:]





Array Mathematics: Broadcasting

Broadcasting in NumPy follows a strict set of rules to determine the interaction between the two arrays:

```
M = np.ones((3, 2))   # M.shape = (3,2)
a = np.arange(3)      # b.shape = (3,)
C=M+a
```

```
ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```

- We see by rule 1 that the array a has fewer dimensions, so we pad it on the left with ones:

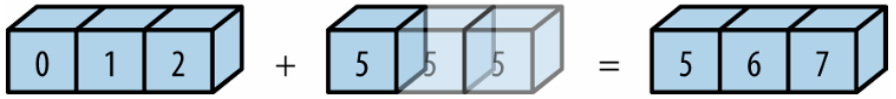
  a.shape -> (3, 2)
  b.shape -> (1, 3)

- By rule 2, we now see that the first dimension disagrees, so we stretch this dimension to match:

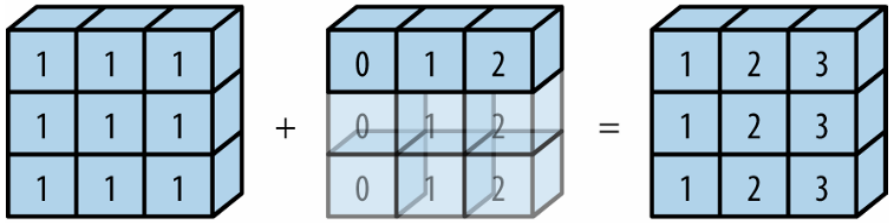
  a.shape -> (3, 2)
  b.shape -> (3, 3)

Visualization of NumPy broadcasting

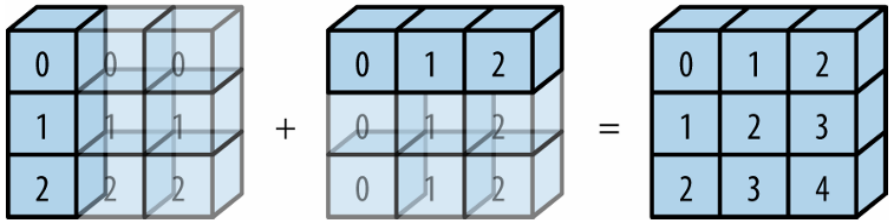
np. arange(3)+5



np. ones((3, 3))+np.arange(3)



np.arange(3)[:,None]+np.arrange(3)[None,:]



Array Mathematics: Broadcasting

Broadcasting is simply a set of rules for applying binary ufuncs (addition, subtraction, multiplication, etc.) on arrays of different sizes.

```
a = np.array([1, 2, 3])
M = np.ones((3, 3))
```

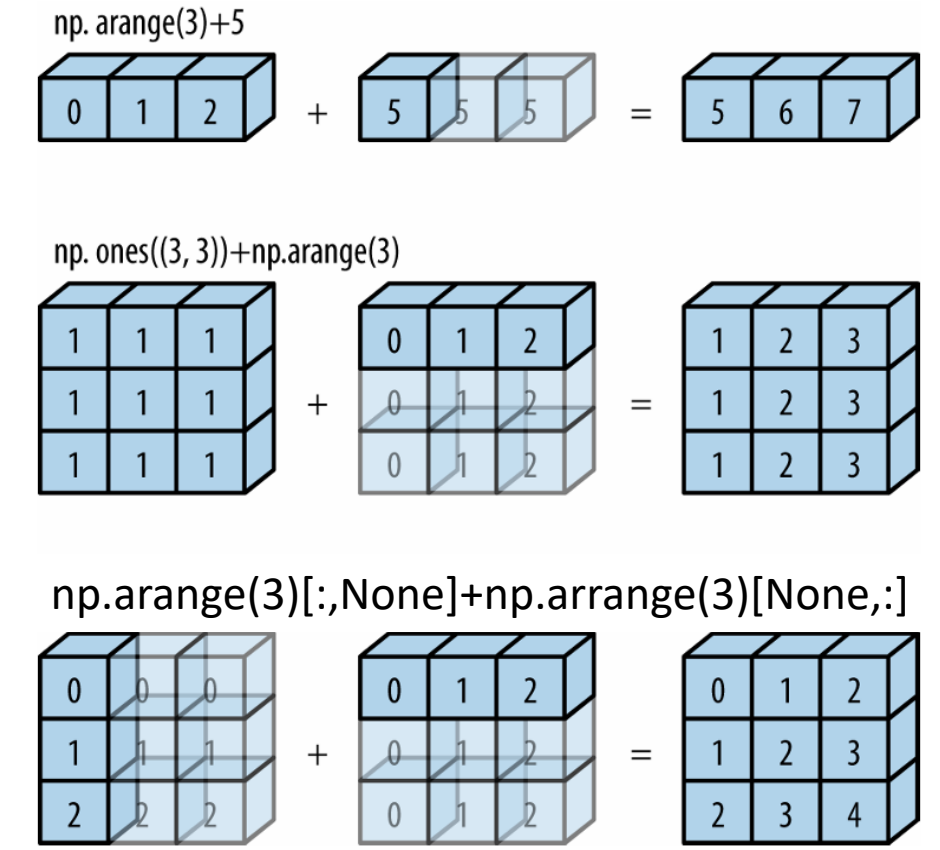
C2=M+a[None,:]

```
array([[2., 3., 4.],
       [2., 3., 4.],
       [2., 3., 4.]])
```

C3=M+a[:,None]

```
array([[2., 2., 2.],
       [3., 3., 3.],
       [4., 4., 4.]])
```

Visualization of NumPy broadcasting



Array Mathematics: Linear Algebra

NumPy provides several linear algebra functions that canbe useful when working with multidimensional arrays. someof these functions include:

- np.dot(A, B): Computes the dot product of two arrays.
- np.linalg.inv(A): Computes the inverse of a square matrix.
- np.linalg.eig(A):Computes the eigenvalues and eigenvectors of a square matrix.
- np.linalg.solve(A, B): soives a linear system of equations.where A is the coefficient matrix and B is the constant matrix.

Remember to always check if your matrices are compatible before performing these operations.

```
A = np.array([[1, 2], [3, 4]]) # Data arrays
B = np.array([[5, 6], [7, 8]]) # Data arrays
dot_product = np.dot(A, B) # Computes the dot product of two arrays
#differ from element-wise multiplication, ele_product=A*B

inverse_A = np.linalg.inv(A) # Computes the inverse of a square matrix
eigenvalues, eigenvectors = np.linalg.eig(A) # Computes the eigenvalues and eigenvectors of a square matrix
solution = np.linalg.solve(A, B) # Solves a linear system of equations
```



Array Analysis: Aggregate Functions

- sum:np.sum(array)-Calculate the sum of all the elements in the array.
- min: np.min(array)- Find the minimum array element.
- max: np.max(array)- Find the maximum array element.“
- mean:np.mean(array)-Calculate the mean of the values in the array.,
- median:np.median(array)- Find the median of the values in the array.
- range: np.ptp(array)-Calculate the range of all the elements in the array.

```
# Creating a sample 1D NumPy array
array_1d = np.array([5, 3, 8, 1, 6, 4])
Array_sum= np. sum (array_1d)
Array_min = np.min (array_1d)

# Creating a sample 2D NumPy array
array_2d = np.array([[5, 3, 8], [1, 6, 4]])
Array_sum= np. Sum (array_2d,  axis=0)
Array_min = np.min (array_2d ,  axis=1)
```

- np.function(your_array,axis=0)

axis=0: the operation will be performed along the rows (axis 0) of the array **(column-wise)**

- np.function(your_array,axis=1)

axis=1: the operation will be performed along the columns (axis 1) of the array **(row-wise)**

Table 2-3. Aggregation functions available in NumPy

Function Name	NaN-safe Version	Description
np.sum	np.nansum	Compute sum of elements
np.prod	np.nanprod	Compute product of elements
np.mean	np.nanmean	Compute median of elements
np.std	np.nanstd	Compute standard deviation
np.var	np.nanvar	Compute variance
np.min	np.nanmin	Find minimum value
np.max	np.nanmax	Find maximum value
np.argmin	np.nanargmin	Find index of minimum value
np.argmax	np.nanargmax	Find index of maximum value
np.median	np.nanmedian	Compute median of elements
np.percentile	np.nanpercentile	Compute rank-based statistics of elements
np.any	N/A	Evaluate whether any elements are true
np.all	N/A	Evaluate whether all elements are true



Array Analysis: Statistical Functions

- std:np.std(array)-Calculate the standard deviation of the values in the array.
- var: np.var(array)-Calculate the variance of the values in the array.

☐ np.function(your_array,axis=0)
axis=0: the operation will be performed along the rows (axis 0) of the array **(column-wise)**
☐ np.function(your_array,axis=1)
axis=1: the operation will be performed along the columns (axis 1) of the array **(row-wise)**

Corrcoef: The np.corrcoef() function in NumPy is used to calculate the correlation coefficient of an array.
The correlation coefficient is a measure of the linear relationship between two variables.

```
# Create a sample 2D NumPy array
array = np.array([[1, 2, 3, 4, 5],
                  [5, 4, 3, 2, 1],
                   [3, 3, 3, 3, 3]])
# Calculate the correlation coefficient of the array
correlation_matrix1 = np.corrcoef(array, rowvar=True)
correlation_matrix2 = np.corrcoef(array, rowvar=False)
```

```
# Creating a sample 1D NumPy array
array_1d = np.array([5, 3, 8, 1, 6, 4])
Array_std= np. std (array_1d)
Array_var = np.var (array_1d)

# Creating a sample 2D NumPy array
array_2d = np.array([[5, 3, 8], [1, 6, 4]])
Array_std= np. std(array_2d,  axis=0)
Array_var = np.var (array_2d ,  axis=1)
```

Array Analysis: Searching Functions

- argmin: np.argmin(array) - Find the index of the minimum array element.
- argmax: np.argmax(array) - Find the index of the maximum array element.
- np.where(condition) - Return the indices of elements in the array that satisfy the given condition.

```
# Creating a sample 1D NumPy array
array_1d = np.array([5, 3, 8, 1, 6, 4])

# Find index of the minimum and maximum array elements
min_index = np.argmin(array_1d)
max_index = np.argmax(array_1d)

# Find indices where elements are greater than 4
indices = np.where(array_1d > 4) [0]

# Computing
New_arry=np.where(a < 5, a, 10*a)
```

- np.function(your_array,axis=0)

axis=0: the operation will be performed along the rows (axis 0) of the array **(column-wise)**

- np.function(your_array,axis=1)

axis=1: the operation will be performed along the columns (axis 1) of the array **(row-wise)**

https://numpy.org/doc/2.2/reference/generated/numpy.where.html



Array Analysis: Searching Functions

- argmin: np.argmin(array) - Find the index of the minimum array element.
- np.argmax(array) - Find the index of the maximum array element.
- np.where(condition) - Return the indices of elements in the array that satisfy the given condition.

```
# Creating a sample 2D NumPy array
array_2d = np.array([[5, 3, 8], [1, 6, 4]])

# Find index of the minimum and maximum array elements in the 2D array
min_index = np.argmin(array_2d,  axis=0)
max_index = np.argmax(array_2d, axis=1)

# Find indices where elements are greater than 4
Indices_i, Indices_j= np.where(array_2d > 4)
```

- np.function(your_array,axis=0)

axis=0: the operation will be performed along the rows (axis 0) of the array **(column-wise)**

- np.function(your_array,axis=1)

axis=1: the operation will be performed along the columns (axis 1) of the array **(row-wise)**



Array Analysis: Sorting Functions

- sort: np.sort(your_array) - Sort the elements in the array in ascending order.
- argsort: np.argsort(your_array) - Returns the indices that would sort the array.

1D array:
a= np.array([3,2,5,4])

	0
0	3
1	2
2	5
3	4

Sort the 1D array in increasing order:

As1=np.sort(a)
As1=a.sort()

	0
0	2
1	3
2	4
3	5

Obtain the sorting index
As1_idx=np.argsort(a)

	0
0	1
1	0
2	3
3	2

Sort the 1D array in descending order:

As2=a[np.argsort(-a)]

	0
0	5
1	4
2	3
3	2

2D array:
b=np.array([[1,4,3], [4,5,1], [2,3,2]])

	0	1	2
0	1	4	3
1	4	5	1
2	2	3	2

Sort the 2D array along the axis 0

B1=b.sort(axis=0)

	0	1	2
0	1	3	1
1	2	4	2
2	4	5	3

Obtain the sorting index
B1_idx=np.argsort(b,axis=0)

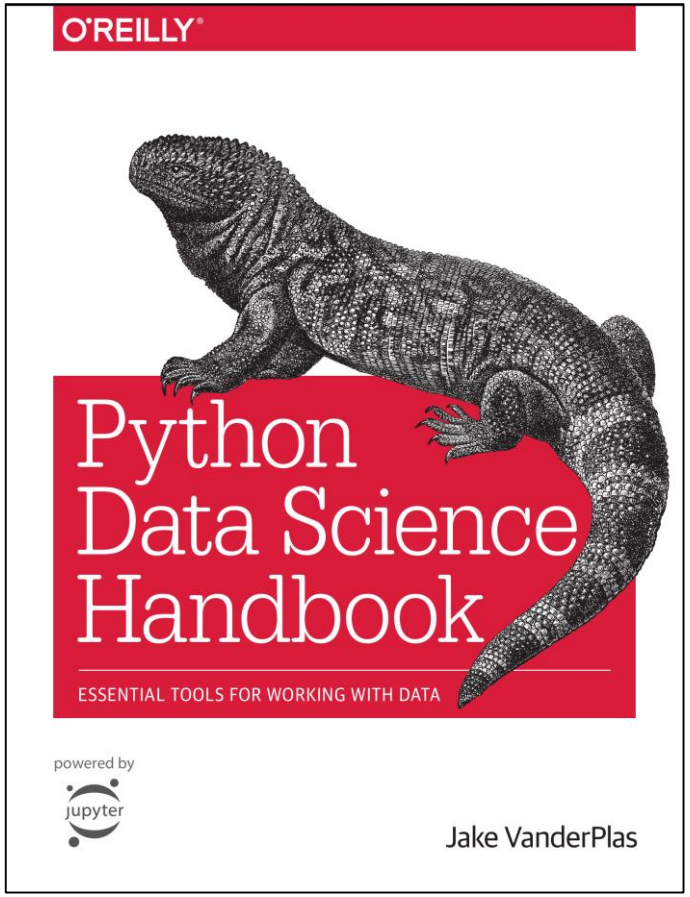
```
[[0, 2, 1],
 [2, 0, 2],
 [1, 1, 0]]
```

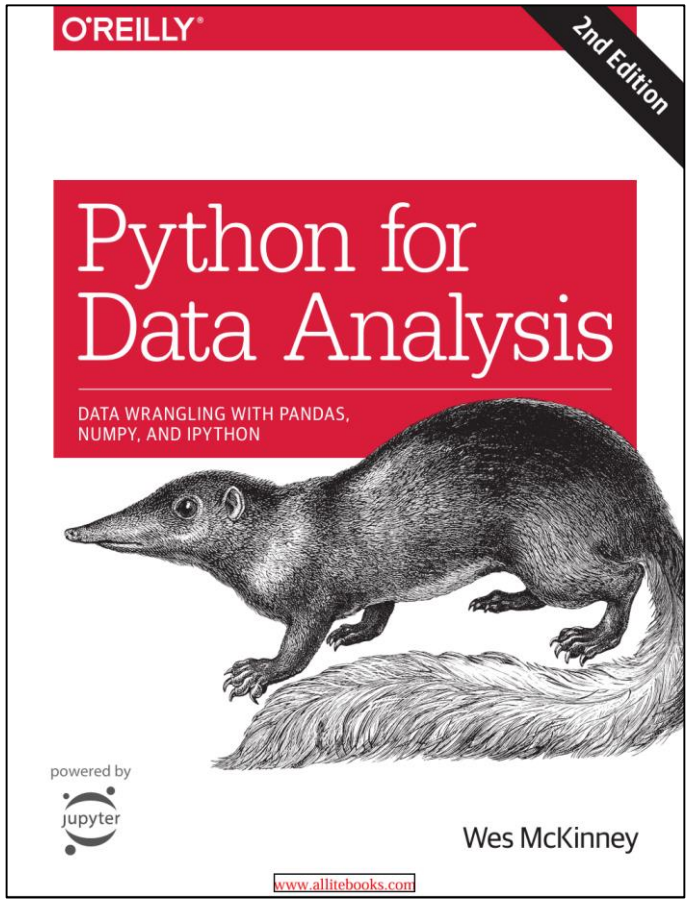
Sort the 2D array along the axis 1

B2=b.sort(axis=1)

?

Extended Reading and Learning





Thanks

